

Robot Tamagotchi & RPG

Technical Design Document (Phase 1)

Architecture & Planning · Status: Draft v1.0 · Prepared for: kaan durukan

This TDD defines the architecture, tech stack, core math, data model, and a phased roadmap for a hybrid idle/RPG game with optional ESP32 hardware integration. No production code yet — this is the blueprint we build against.

1. Executive Summary & Design Principles

We are building "RoboPet" (working title): an offline-first, single-player-with-async-PvP mobile game that fuses three proven loops — idle/Tamagotchi care, gacha collection, and turn-based RPG combat — with a differentiating optional physical ESP32 companion.

The single most important architectural constraint is this:

The hardware is an accessory, never a dependency. The game must be 100% complete and playable through the touchscreen alone. The ESP32 is a presence/ambient layer that mirrors and conveniences existing in-app actions. Hardware can never be the source of truth for game state — the app (and its cloud backup) always is.

Guiding principles:

- Offline-first / local-authoritative. The device local DB is the working source of truth; cloud is for backup, anti-cheat validation, and async PvP.
- Deterministic, server-validatable simulation. Idle progression is computed from timestamps + rates, not from a constantly-running timer. Cheap, resumable, cheat-resistant.
- Separation of simulation from rendering. Game state advances independently of the UI frame loop. The UI subscribes to state; it never is the state.
- Additive, data-driven content. Robots, modules, loot tables, and puzzles are defined in data (JSON/remote config), not hardcoded, so we can balance and add content without app updates.

2. Recommended Tech Stack

2.1 Mobile framework — recommendation: Flutter

Both Flutter and React Native are viable. For this project I recommend Flutter, with the reasoning laid out transparently below so you can override if your team expertise points elsewhere.

Criterion	Flutter	React Native	Why it matters here
Custom animated game UI	Strong — Skia/Impeller, 60–120fps, Flame engine	Adequate — heavier for custom canvas	A pet/RPG game lives on UI feel
BLE support	Mature (flutter_blue_plus)	Mature (react-native-ble-plx)	Both fine
Offline DB options	Excellent (Isar, Drift, ObjectBox)	Good (WatermelonDB, Realm)	Need a fast reactive local DB
Codebase performance	Native ARM , no JS bridge	JS runtime + JSI bridge	Idle sim + combat math
Embedded 2D engine	Flame (purpose-built)	No first-class equivalent	Combat scene & animation

Verdict: Flutter + Flame for the combat/pet rendering scenes, with standard Flutter widgets for menus, inventory, and gacha UI. If your team is already deep in TypeScript/React, React Native is acceptable — the rest of this document is framework-agnostic.

2.2 Architecture pattern

Adopt a layered, unidirectional architecture:

```

Presentation (Widgets / Flame scenes)
  | (watches state, sends intents)
  v
Application / State Management ---> Riverpod (recommended) or Bloc
  | (use-cases: FeedRobot, OpenLootBox, ResolveBattle)
  v
Domain (pure Dart: entities, simulation engine, combat resolver, RNG)
  |
  v
Data (Repositories) ---> Local DB (Isar) + Remote (Firebase) + Hardware (BLE service)

```

- State management: Riverpod (testable, compile-safe providers) or flutter_bloc for explicit event/state machines. The combat system is a great fit for a finite state machine.
- Domain layer is pure Dart — no Flutter imports. The idle-simulation engine and combat resolver must be unit-testable in isolation and portable to a Cloud Function so the server can re-run the same logic for anti-cheat validation.

2.3 Local offline-first database — recommendation: Isar

Option	Model	Best for	Notes
Isar	NoSQL object DB, reactive queries	Recommended — fast, typed, nested robot objects	Reactive watch() pairs with Riverpod
Drift	SQLite + type-safe SQL	Relational queries / local leaderboards	More boilerplate, more power
Hive	Lightweight key-value	Settings / simple flags only	Too simple for the full robot graph

Pattern: local DB holds the authoritative live state. On meaningful change (feed, battle end, loot open) and on app pause, debounce-write to the DB and enqueue a cloud sync.

2.4 Backend / cloud — recommendation: Firebase

- Firebase Auth — anonymous auth first (frictionless), upgradeable to Google/Apple sign-in for cross-device save.
- Cloud Firestore — player save backup + async PvP "ghost" snapshots (you battle a snapshot, not a live player).
- Cloud Functions — authoritative for gacha pulls (server-side RNG), reward grants, and anti-cheat re-simulation of idle gains.
- Remote Config — live-balance loot tables, idle rates, drop rarities without shipping an app update.
- (Optional) FCM — push notification "Your robot battery is low!" to re-engage players.

2.5 ESP32 communication — recommendation: BLE (primary)

With an optional Wi-Fi/Firebase bridge as an advanced secondary option. Full analysis in Section 5.

3. State Management & The Core Idle Loop (The Math)

This is the heart of an idle game. Do NOT run a setInterval/Ticker that decrements stats in real time — it is battery-hungry, wrong when the app is killed, and trivially cheatable. Instead use timestamp-delta simulation.

3.1 The principle

Each decaying stat stores its last value and the timestamp it was last computed. The current value is derived on demand from elapsed wall-clock time.

Core formula (linear decay): $S_{\text{now}} = \text{clamp}(S_{\text{last}} - R_{\text{decay}} \times \Delta t, 0, S_{\text{max}})$ where $\Delta t = (t_{\text{now}} - t_{\text{last}})$ in seconds, and R_{decay} is the per-second decay rate for that stat.

3.2 "Catch-up" on app resume

When the app reopens (or comes to foreground), run a single reconciliation pass:

```
void reconcile(Robot r, DateTime now) {
    final dtSeconds = now.difference(r.lastTickAt).inMilliseconds / 1000.0;
    if (dtSeconds <= 0) return; // guard against clock going backwards

    // Decay care stats (offline progression)
    r.battery      = clamp(r.battery      - r.batteryDrain * dtSeconds, 0, r.batteryMax);
    r.oil          = clamp(r.oil          - r.oilDrain     * dtSeconds, 0, r.oilMax);
    r.entertainment = clamp(r.entertainment - r.funDrain    * dtSeconds, 0, r.funMax);

    // Accrue idle currency (capped to discourage neglect / infinite hoarding)
    final earnSeconds = min(dtSeconds, r.idleEarnCapSeconds); // e.g. 8h cap
    r.scrapCurrency += (r.scrapPerSecond * earnSeconds).floor();
}
```

```

    _applyNeglect(r, dtSeconds); // penalty if a stat hit zero during the gap
    r.lastTickAt = now;
}

```

3.3 Foreground smoothing

While the app is open, you still want the bars to visibly tick down. Use a lightweight UI ticker (once per second, or a Flame update loop) that calls the SAME pure decay function — never a second implementation. One source of truth for the math, called both on resume and on each visible frame.

3.4 Guard rails (game-feel + anti-exploit)

- Decay cap / grace period: optionally freeze decay after N hours so a returning player is not punished with a "dead" robot.
- Idle earnings cap (`idleEarnCapSeconds`): currency only accrues up to e.g. 8 hours offline.
- Clock-tamper detection: compare device time against a trusted source (Firestore server timestamp / NTP). If the clock jumped forward unrealistically, clamp the delta and flag the save. Never grant high-value rewards based purely on client-reported elapsed time.
- Monotonic guard: if `now < lastTickAt`, do not advance (handles timezone/clock rollback).

3.5 Non-linear options (for later tuning)

Linear decay is the right starting point. Later you may want curved rates, e.g. exponential approach for "charge" mechanics: $S_{\text{now}} = S_{\text{target}} + (S_{\text{last}} - S_{\text{target}}) \times e^{(-k\Delta t)}$. Keep these behind the same single decay interface so balancing is a data change, not a refactor.

4. Data Model

Draft schema for the core entities. Designed to be (a) serializable to JSON for cloud sync, (b) storable as Isar objects, and (c) data-driven (definitions vs. instances separated).

4.1 Key modeling decision: Definition vs. Instance

- Definitions (`RobotSpecies`, `ModuleDef`, `LootTable`) are static content — shipped/remote-config, identical for all players, referenced by ID.
- Instances (`RobotInstance`, `PlayerProfile`) are mutable per-player state.

This keeps saves small (store IDs + deltas, not full stat blocks) and lets you re-balance definitions remotely.

4.2 RobotInstance (the entity you asked for)

```

{
  "instanceId": "rbt_a1b2c3d4",
  "speciesId": "rover_scout",           // FK -> RobotSpecies definition
  "nickname": "Sparky",
  "rarity": "RARE",                     // COMMON | RARE | EPIC | LEGENDARY
  "class": "SCOUT",                     // SCOUT | TANK | BRAWLER | SUPPORT
  "level": 7, "xp": 1340, "xpToNext": 1800,

```

```

"careStats": {
  "battery":      { "value": 72.5, "max": 100, "drainPerSec": 0.0008 },
  "oil":          { "value": 40.0, "max": 100, "drainPerSec": 0.0005 },
  "entertainment": { "value": 88.0, "max": 100, "drainPerSec": 0.0011 }
},

"combatStats": { "hp": 320, "attack": 54, "defense": 38, "speed": 47, "critChance": 0.08
},

"equippedModules": {
  // slot -> moduleInstanceId (null = empty)
  "wheels": "mod_mecanum_t2_x91", "armor": "mod_plating_t1_a02",
  "sensor": null, "core": "mod_powercore_t2_c14", "utility": null
},

"abilities": ["ability_ram", "ability_overclock"],

"timestamps": {
  "createdAt": "2026-06-04T18:30:00Z",
  "lastTickAt": "2026-06-04T21:15:42Z", // drives offline reconciliation
  "lastFedAt": "2026-06-04T20:50:10Z"
},

"flags": { "isActive": true, "isFavorite": false, "neglectStrikes": 0 },
"schemaVersion": 1 // for save migrations
}

```

4.3 ModuleInstance & ModuleDef

```

// Instance (owned by player)
{
  "moduleInstanceId": "mod_mecanum_t2_x91",
  "defId": "mecanum_wheels", "tier": 2, "slot": "wheels",
  "statModifiers": { "speed": 12, "defense": -2 }, // flat or %
  "cosmeticVariant": "chrome"
}

// Definition (static content)
{
  "defId": "mecanum_wheels", "displayName": "Mecanum Wheels", "slot": "wheels",
  "appearanceKey": "wheel_mecanum", // drives sprite/3D swap
  "maxTier": 5, "baseModifiers": { "speed": 6 }, "perTierModifiers": { "speed": 3 },
  "upgradeCost": { "scrap": 200, "rareCogs": 2 }
}

```

Derived (effective) stat = base combat stat + sum of flat module modifiers, then x (1 + sum of percent modifiers). Compute in a pure function; never persist the derived value — recompute it so re-balancing modules is automatic.

4.4 PlayerProfile

```

{
  "playerId": "firebase_uid_xyz", "displayName": "kaan",
  "currencies": { "scrap": 4820, "premiumGems": 35, "rareCogs": 12 },
  "ownedRobotIds": ["rbt_a1b2c3d4", "rbt_99ff00"],

```

```

"ownedModuleIds": ["mod_mecanum_t2_x91", "mod_plating_t1_a02"],
"activeRobotId": "rbt_a1b2c3d4",
"lootBoxes": { "common": 2, "rare": 1, "epic": 0 },
"lootTimer": { "nextFreeBoxAt": "2026-06-04T23:00:00Z" },
"pvp": { "rank": "Bronze III", "trophies": 240 },
"settings": { "hardwarePaired": false, "pairedDeviceId": null },
"schemaVersion": 1
}

```

4.5 Gacha / Loot Box model

```

{
  "lootTableId": "epic_box_v3",
  "pity": { "epicGuaranteedAtPull": 30 }, // pity counter to bound bad luck
  "drops": [
    { "type": "robot", "speciesId": "rover_scout", "rarity": "COMMON", "weight": 60 },
    { "type": "robot", "speciesId": "tank_bulwark", "rarity": "RARE", "weight": 28 },
    { "type": "robot", "speciesId": "brawler_volt", "rarity": "EPIC", "weight": 12 }
  ]
}

```

Gacha pulls must be resolved server-side (Cloud Function). Client-side RNG on an offline save is trivially exploitable. The client requests a pull; the server rolls against the authoritative loot table + pity counter and returns the result — and keeps loot weights out of the client binary.

4.6 Save migration

Every persisted root object carries schemaVersion. On load, run an ordered migration chain (v1 -> v2 -> v3). Non-negotiable for a live game — you WILL change the schema after launch.

5. ESP32 Integration — BLE vs. Wi-Fi (Full Analysis)

5.1 The use cases

1. Low-battery alert: virtual battery low -> physical LEDs blink.
2. Physical feed: player presses tactile button -> robot recharges in-app.

Both are low-bandwidth, proximity-based, ambient-desk interactions. That profile points strongly to one answer.

5.2 Recommendation: BLE as primary transport

Factor	BLE (Bluetooth Low Energy)	Wi-Fi / Firebase bridge
Setup friction	Low — pair from app, no router credentials	High — must provision SSID/password to a headless device
Power draw (desk gadget)	Very low — deep-sleep + advertise	Higher — Wi-Fi radio is power-hungry

Button-press latency	Low, direct phone <-> device	Round-trips through cloud unless on same LAN
Works without internet	Yes (aligns with offline-first)	No (cloud path) / LAN-only otherwise
Remote (away) sync	No — proximity only	Yes — works anywhere

Why BLE wins: the entire fantasy is "a little robot on my desk that reacts to me right now." That is a proximity interaction — lowest setup friction, lowest power, no internet dependency, direct low-latency button events. Wi-Fi only advantage (control from anywhere) is not part of the core fantasy.

When to add Wi-Fi/Firebase (Phase 4+, optional): if you later want the device to react while the player is away (LED pulses when a Cloud Function detects low battery), add an optional Wi-Fi mode where the ESP32 subscribes to the player Firestore/RTDB node. Treat as an advanced opt-in, not the default.

5.3 BLE design (GATT profile sketch)

```
Service: RoboPet (custom 128-bit UUID)
+-- batteryLevel    [WRITE from phone] -> uint8 0-100; firmware maps to LED behavior
+-- ledCommand      [WRITE from phone] -> enum {OFF, SOLID, BLINK_SLOW, BLINK_FAST}
+-- buttonEvent     [NOTIFY to phone]  -> sends on each tactile press
+-- deviceStatus    [READ]             -> firmware version, gadget battery
```

- Phone -> device: when reconciliation computes battery < threshold, write ledCommand = BLINK_FAST. When recharged, write OFF.
- Device -> phone: firmware sends a buttonEvent NOTIFY on press; the app BLE service translates it into the SAME FeedRobot use-case the on-screen button triggers.

Critical decoupling: the physical button and the on-screen button must both dispatch the identical domain intent (FeedRobot). The hardware layer is just another input source. This is what makes the hardware genuinely optional — remove it and nothing in the game logic changes.

5.4 Firmware responsibilities (ESP32, Arduino/ESP-IDF)

- Advertise the GATT service; handle pairing/bonding.
- Drive LEDs based on last-written ledCommand (with a default idle animation when disconnected).
- Debounce the tactile button; send buttonEvent notifications.
- Deep-sleep between events to conserve power; wake on BLE/button.
- Hold no authoritative game state — only reflect what the app tells it and report raw input.

5.5 Graceful degradation (the optional contract)

- App boots and plays fully with hardwarePaired = false.
- A "Connect Device" screen is opt-in, tucked in settings.
- All hardware calls are wrapped so a disconnect, denied permission, or missing device is a no-op, never a crash.
- Any reward obtainable via the physical button is equally obtainable via the on-screen button.

6. Combat System (RPG) — Design Direction

Recommendation: turn-based auto-battler for v1. It fits idle/collection games, requires no real-time netcode, and enables asynchronous PvP cleanly.

- Deterministic resolver: combat is a pure function `resolve(teamA, teamB, seed) -> BattleResult`. Same inputs + seed always produce the same outcome.
 - Why deterministic + seeded: (a) the client renders the fight as animation while the outcome is already decided; (b) the server can re-run the exact fight from the seed to validate PvP (anti-cheat); (c) battles are replayable/shareable.
 - Async PvP via ghosts: you fight a snapshot of the opponent team stored in Firestore. No matchmaking server, no live connection.
 - Turn order driven by speed; $\text{damage} = f(\text{attack}, \text{enemy defense}, \text{crit}, \text{ability modifiers})$. Keep the formula data-driven for balancing.
-

7. Logic Minigame (Extreme-Sudoku-Inspired)

- Self-contained module with its own generator + solver + validator, decoupled from the rest of the game (it only emits a "reward grant" event on completion).
 - Generation: generate a solved grid, then remove cells while a unique-solution solver confirms exactly one solution remains. Difficulty = number/pattern of removed givens + which advanced techniques are required.
 - Validation: never trust the client "I solved it." For rare-material rewards, validate the submitted solution server-side before granting.
 - Variants for "extreme": killer-sudoku cages, thermo/arrow constraints, or sandwich clues — each is an additional constraint checker layered on the base validator.
-

8. Development Roadmap

Where to start: the pet care loop — NOT the ESP32, NOT combat. Build the smallest thing that proves the core fantasy and the hardest-to-retrofit system (offline simulation + persistence + save migration). Hardware and combat plug into this foundation later. Building hardware first is the classic trap.

Phase 0 — Foundations (Week 1)

- Repo, CI, project scaffold, layered architecture skeleton.
- Pure-Dart domain layer stub + unit test harness.
- Isar local DB wired up with schemaVersion + migration scaffold.

Phase 1 — The Living Pet (Weeks 2–4) [START HERE]

- RobotInstance model + persistence.
- Timestamp-delta decay engine (Section 3) with full unit tests for offline catch-up, caps, and clock-tamper guards.
- One robot, three care stats, feed/clean/play actions via on-screen buttons.
- Basic UI: animated stat bars, a placeholder robot sprite, app-resume reconciliation.
- Exit criteria: close app for an hour, reopen, stats correctly reflect elapsed time.

Phase 2 — Collection & Progression (Weeks 5–8)

- Firebase Auth (anonymous) + Firestore save backup + sync queue.
- Server-side gacha Cloud Function + loot tables in Remote Config + pity system.
- Multiple owned robots; active-robot switching.
- Modules: equip/unequip, derived-stat computation, appearance swaps.

Phase 3 — Combat (Weeks 9–12)

- Deterministic seeded combat resolver (pure Dart) + full test suite.
- Flame-based battle scene rendering the resolved fight.
- PvE ladder first; then async PvP ghosts in Firestore + server re-simulation.

Phase 4 — Minigame & Economy Depth (Weeks 13–15)

- Sudoku-style generator/solver/validator module + reward hook.
- Server-side solution validation for rare-material grants.
- Economy tuning pass via Remote Config.

Phase 5 — ESP32 Hardware Layer (Weeks 16–18) — optional, built last

- BLE service in-app behind the optional "Connect Device" flow.
- GATT profile + ESP32 firmware (LED control + button NOTIFY).
- Wire physical button -> existing FeedRobot intent; battery threshold -> ledCommand.
- Verify full graceful degradation (every hardware path is a safe no-op when unpaired).

Phase 6 — Polish, Telemetry, Live-Ops (Weeks 19+)

- FCM re-engagement pushes, analytics, crash reporting, A/B balance via Remote Config, store assets.

9. Cross-Cutting Concerns & Risks

Concern	Mitigation
Save tampering / cheating	Server-authoritative gacha, server re-sim of PvP & high-value idle gains, clock-tamper detection
Schema evolution post-launch	schemaVersion + ordered migration chain on every root object from day one
Hardware becoming a dependency	Strict decoupling: hardware = input source feeding existing intents; no-op when unpaired

Balancing without app updates	All rates, loot weights, costs in Firebase Remote Config, not the binary
Battery drain from "always running"	No real-time timers; timestamp-delta simulation only
Determinism for combat replay	Pure, seeded resolver portable to Cloud Functions

10. Open Questions (to finalize before Phase 1 code)

3. Framework lock-in: confirm Flutter, or is your team expertise pushing toward React Native?
4. Monetization model: is premiumGems a real IAP currency (affects gacha/server design), or purely earned?
5. PvP scope for v1: async ghost battles only (recommended), or is live PvP a hard requirement?
6. Art pipeline: 2D sprite-based robots (faster, fits Flame) or 3D models? Affects the appearanceKey module-swap system.
7. Target platforms: iOS + Android both at launch? Minimum OS versions? (Affects BLE API choices.)

Next step: once you confirm the open questions above (especially the framework), we move to the Phase 1 detailed spec — exact Isar schemas, the decay engine implementation, and the test matrix — then start writing production code for the living-pet loop.